Git Workflow Framework for Feature Development and Code Review Submission

Overview

This document outlines a structured Git workflow designed to manage feature development, branch synchronization across remotes, and code submission for review. The approach assumes the existence of a central repository (R), a forked repository (F), and a local development environment (L). It emphasizes maintaining clean branch histories, enabling effective collaboration and review.

Initial Setup (T1)

1. **Clone the central repository (R)**

```
git clone https://github.com/UpstreamOrg/myrepo.git
```

1. **Rename the default remote** `** to **`

```
git remote rename origin Upstream
```

1. **Create a local tracking branch** `** from **`

```
git checkout -b UMaster Upstream/master
```

1. **Add the forked repository (F) as a remote**

```
git remote add Review https://github.com/YourForkOrg/myrepo.git
```

1. **Create a local tracking branch** `** from **`

```
git fetch Review
git checkout -b RMaster Review/master
```

Feature Development (T2)

1. **Check out the latest** ``

```
git checkout RMaster
```

1. **Create a local feature branch** `** from **`

```
git checkout -b B
```

1. **Develop features and commit changes**

```
git add .
git commit -m "Implement feature XYZ"
```

At this point, branch `B` contains unreviewed feature work and is not yet linked to any remote.

Iterative Review and Sync Loop

Algorithm:

```
idx = 1
flag = true
while (flag) {
    Phase 1:
        Update UMaster and rebase RMaster
        Rebase feature branch B on updated RMaster
        Push feature branch to Review as B1-V<idx>
        Submit B1-V<idx> for review

    Phase 2:
        if (review is approved)
            flag = false
            Merge or push to Upstream
        else
            Apply review changes
            idx++
}
```

Example Commands:

Phase 1: Syncing and Submission

1. **Ensure** `** is up to date with **`

```
git checkout UMaster
git pull --rebase
```

1. **Rebase `** with **` to sync remotes**

```
git checkout RMaster
git rebase UMaster
git push Review RMaster
```

1. **Rebase feature branch `** on **`**

```
git checkout B
git rebase RMaster
```

1. **Push feature branch to `** as **`**

```
git push Review B:B1-V1
```

1. **Submit branch `` for review**

Phase 2: Handling Review Feedback

1. **If review is approved**

```
git checkout B
git rebase UMaster
git push Upstream B
```

1. **If changes are requested, amend and re-push**

```
git commit --amend  # Or make additional commits
git rebase RMaster
git push Review B:B1-V2
```

Supporting Commands

- **Find current remotes**

```
git remote -v
```

- **Rename a remote**

```
git remote rename <old-name> <new-name>
```

- **Add a new remote**

```
git remote add <name> <url>
```

- **Create a local branch tracking a remote branch**

```
git checkout -b <local-branch> <remote>/<branch>
```

Benefits of This Workflow

1. **Clarity of Branch Origins**: Separate local branches (`UMaster`, `RMaster`) clearly distinguish between upstream and forked states.
2. **Safe Collaboration**: Rebasing forked branches on upstream before pushing ensures a linear, clean history and minimizes merge conflicts.
3. **Review Traceability**: Naming review branches as `B1-V<idx>` offers an intuitive version history for reviewers.
4. **Modular Setup**: Easily replicable and extendable to projects with multiple remotes.
5. **Automation Friendly**: The structure can be codified into scripts for consistent execution.

Conclusion

This workflow provides a robust framework for managing Git branches across multiple remotes with iterative development and review. It encourages best practices in history hygiene, collaboration, and modular project maintenance.